

Wedge

브라우저 실행 기반 UX 진단 리포트 서비스

개발 기간 2026.04 ~ 2026.05 약 7주

플랫폼 Web

개발 인원 6명

담당 역할 Spring Backend

- 분석 진행 상태 연동
- Analysis Outbox/Retry
- Analyzer 병렬 처리 검증

개발 환경

언어 Java 17 / TypeScript / Python

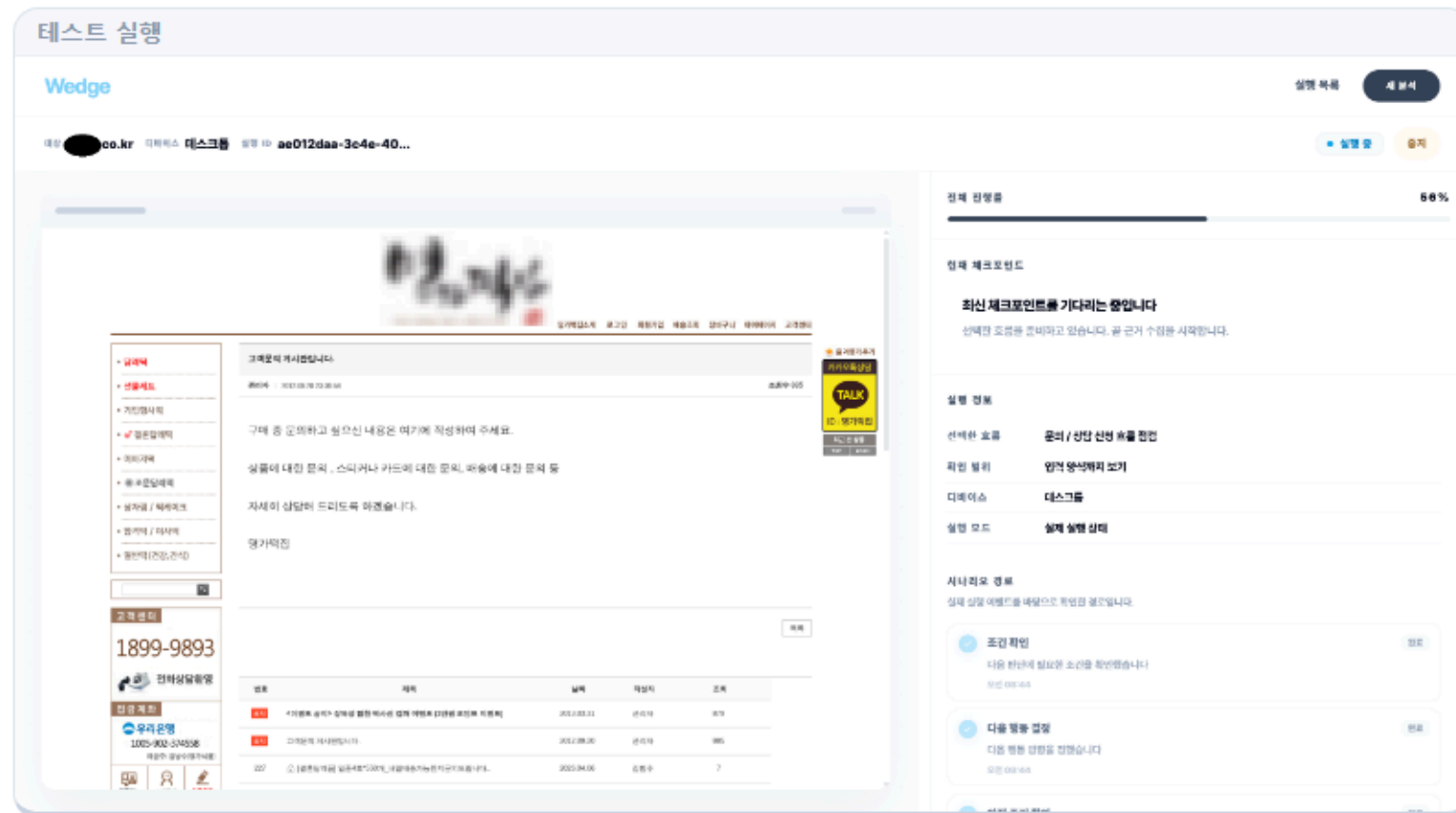
서버 Spring Boot 3.5.14 / MyBatis

프론트 React / Vite

Runner Node.js 24 / Playwright

DB / MQ PostgreSQL 17 / Redis 8.6 / RabbitMQ

인프라 Docker / Nginx / Jenkins / S3



{

세부 구현사항 1

분석 진행 상태 연동

- 브라우저 실행기가 보내는 진행 결과 알림을 Spring 서버에서 수신했습니다.
- 실행 단계별 상태를 DB 기준 상태로 반영했습니다.
- 사용자가 보는 진행 상황과 실제 처리 상태가 어긋나지 않도록 연결했습니다.

Analysis Outbox/Retry

- 분석 Job 상태 변경과 outbox 메시지 저장을 하나의 DB 트랜잭션 안에서 처리했습니다.
- MQ 발행은 트랜잭션 커밋 이후 실행되도록 분리해, 커밋 전 발행되는 상황을 피했습니다.
- 발행 실패 시 outbox에 실패 상태와 다음 시도 시점을 남겨 재전달 가능한 흐름으로 구성했습니다.

}

분석 요청이 처리되는 흐름

사용자의 분석 요청이 저장되고, 작업 큐를 거쳐 브라우저 실행·분석 처리 결과가 다시 상태로 반영되는 구조입니다.

사용자 분석 요청

웹 화면에서 URL과 분석 조건 입력



Spring API Server

요청 저장 · 실행 상태 생성



Analysis Outbox

전달 대기 작업 기록 · 재시도 기준



작업 큐

브라우저 실행기와 분석 처리기로 작업 전달



브라우저 실행 · 분석 처리

화면 실행, 캡처, 판단 결과 생성



진행 상태 업데이트

진행/완료/실패 상태를 서버 DB에 반영

- 핵심은 요청 저장, 작업 전달, 진행 상태 반영이 중간에 끊기지 않도록 연결한 것입니다.

{ 세부 구현사항 2

Analyzer 병렬 처리 적용

- 리포트 생성 중 오래 걸리는 이미지 판단 요청을 일부 병렬로 처리했습니다.
- 한 번에 너무 많은 요청이 몰리지 않도록 동시 처리 수를 제한했습니다.
- 처리 방식별 실행 시간을 기록해 개선 과정을 비교했습니다.

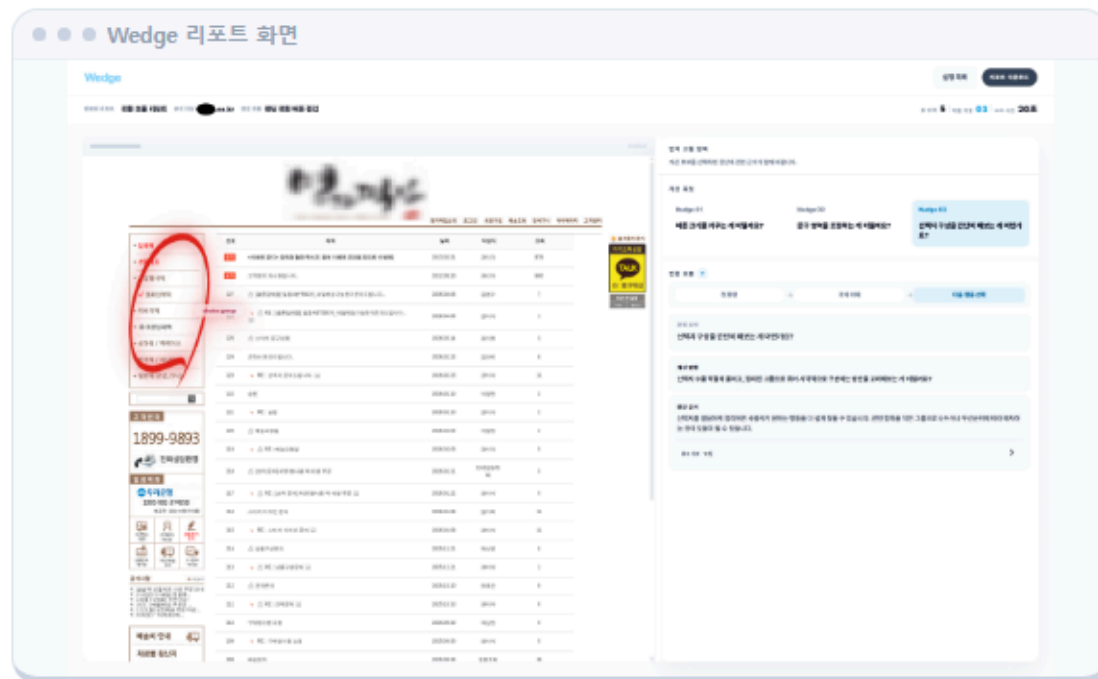
결과 안정성 검증

- 순차 처리와 병렬 처리의 판단 결과가 같은지 비교했습니다.
- 속도 개선이 결과 품질을 해치지 않는지 확인했습니다.
- 실패 요청이 숨겨지지 않도록 예외 흐름과 측정 조건을 함께 남겼습니다.

}

리포트 생성 대기 시간 개선

사용자가 분석 완료를 기다리는 시간을 줄이기 위해, 오래 걸리는 이미지 판단 구간을 병렬 처리하고 결과 동일성을 확인했습니다.



/ 분석 결과 화면 / 리포트 생성 완료

첫 번째 개선

84.0s → 63.5s

화면 3개를 일부 동시에 확인
약 24.4% 단축

두 번째 개선

63.5s → 42.3s

63.5초 기준 환산
약 33.4% 단축

{

문제 해결과 회고

문제 해결 관점

- 분석 상태를 대기 상태로 바꾼 뒤 MQ 발행이 실패하면, 화면 상태와 실제 작업 전달 상태가 어긋날 수 있었습니다.
- 상태 변경과 **outbox 저장**은 DB 트랜잭션으로 함께 처리하고, MQ 발행은 커밋 이후 단계로 분리했습니다.
- 발행 실패 기록과 재시도 기준을 남겨 작업이 조용히 사라지지 않도록 보완했습니다.
- **Analyzer** 병렬 처리 개선은 속도 수치뿐 아니라 결과 동일성과 실패 흐름을 함께 확인했습니다.

프로젝트 회고

역할이 나뉜 서비스에서는 각 영역이 주고받는 데이터 기준을 명확히 해야 변경 리스크를 줄일 수 있었습니다. 또한 사용자가 보는 진행 흐름과 서버 내부 상태가 어긋나지 않도록 맞추는 과정이 백엔드 구현의 핵심이라는 점을 배웠습니다.

}

